MPHASIS TECHNICAL INTERVIEW MASTER GUIDE

Core Concepts, Coding Solutions, SQL Queries & System Architecture • Batch of 2026

Overview: Compiled directly from candidate feedback across Mphasis Round 2 Technical Interviews (Batch of 2026). Covers OOPs, Data Structures & Algorithms, Live Coding, SQL Queries, SDLC/Agile, Web Tech, Resume Deep Dives & Stress Scenarios.

CATEGORY 1: TECHNICAL INTRODUCTIONS & JOURNEY

Q1 Introduce yourself technically and walk me through your engineering journey.

RECOMMENDED TECHNICAL RESPONSE:

"Good morning. I am [Your Name], a final-year student pursuing [Degree] at MITE with an aggregate CGPA of [Your CGPA]. Technically, I specialize in object-oriented programming using Java/Python, relational database design with SQL, and web technologies. My journey began with core algorithmic logic in early semesters, progressing to full-stack project development and industry internships where I applied software engineering principles. I have actively participated in hackathons and upskilled via technical certifications. I am eager to apply my technical baseline at Mphasis."

Q2 Talk about yourself for 3 minutes without mentioning your college, school, marks, or project details.

RECOMMENDED STRATEGY & RESPONSE:

"I am a tech-enthusiast driven by structured problem-solving and clean logic. My core identity centers around being a continuous learner who enjoys analyzing how complex systems work under the hood. As an individual, I value systematic organization, data-driven reasoning, and adaptability. When faced with unknown technical challenges, my default approach is to break down the problem into smaller components, consult documentation, test hypotheses systematically, and iterate toward a solution. Beyond coding, I enjoy peer mentoring and strategic gaming, which sharpens my spatial reasoning and focus under time constraints."

> *Tip: This tests spontaneous thinking, self-awareness, and verbal fluency under constraint rules.*

Q3 What do people actually do in the IT Industry day-to-day?

RECOMMENDED RESPONSE:

"IT professionals build, maintain, and secure digital software infrastructure that powers global enterprises. Day-to-day work involves:

- **Agile Ceremonies:** Daily stand-ups, sprint planning, and backlog refinement.
- **Development & Refactoring:** Writing modular code, developing REST APIs, and implementing business logic.
- **Quality Assurance & Testing:** Writing unit tests, code reviews, and automated integration testing.
- **Deployment & Support:** Monitoring CI/CD pipelines, cloud infrastructure management, and fixing production bugs.

"

CATEGORY 2: OBJECT-ORIENTED PROGRAMMING (OOPS)

Q4 What are the core principles of OOPs? Explain with real-world examples.

RECOMMENDED RESPONSE:

The four core pillars of Object-Oriented Programming are:

- **Encapsulation:** Bundling data (variables) and methods into a single unit (class) while restricting direct access using private access modifiers (e.g., Bank Account class hiding account balance behind getBalance()/deposit() methods).
- **Abstraction:** Hiding internal complexity and showing only essential features (e.g., A car's accelerator pedal—you press it to speed up without needing to see the internal fuel injection engine mechanism).
- **Inheritance:** Allowing a child class to inherit properties and methods from a parent class (e.g., Vehicle class extended by ElectricCar class).
- **Polymorphism:** The ability of a message or method to take multiple forms—Method Overloading (compile-time) and Method Overriding (runtime).

Q5 What is the difference between Method Overloading and Method Overriding?

RECOMMENDED COMPARISON:

- **Method Overloading (Compile-time Polymorphism):** Occurs within the same class when multiple methods share the same name but have different parameter lists (number, order, or types of parameters). Return type can differ, but parameters must change.
- **Method Overriding (Runtime Polymorphism):** Occurs between parent and child classes when a child class provides a specific implementation of a method already defined in its parent class. Method signature (name, parameters, return type) must be identical.

Q6 What is the difference between a Class and a Structure?

RECOMMENDED RESPONSE:

- **Class:** Reference type (stored on Heap). Supports full OOP features like inheritance, default access specifiers are `private`, and can contain constructors/destructors.
- **Structure (Struct):** Value type (stored on Stack). Used mainly for lightweight data grouping. Default access specifiers are `public`, and generally does not support inheritance.

Q7 What is the difference between a Normal Variable and a Static Variable?

RECOMMENDED RESPONSE:

- **Normal (Instance) Variable:** Created when an object is instantiated and destroyed when the object is garbage collected. Each object retains its own separate copy of instance variables.
- **Static Variable:** Belongs to the class itself rather than any specific instance. Only one copy exists in memory shared among all objects of that class. Created when class is loaded.

CATEGORY 3: PROGRAMMING LANGUAGES (JAVA, PYTHON, C)

Q8 What are Java Data Types? Explain Primitive vs Non-Primitive.

RECOMMENDED RESPONSE:

- **Primitive Data Types:** Predefined by the language. Holds raw values directly in memory. Java has 8 primitives: `byte` (1B), `short` (2B), `int` (4B), `long` (8B), `float` (4B), `double` (8B), `char` (2B), and `boolean` (1 bit).
- **Non-Primitive Data Types:** Created by programmers (reference types). Store memory addresses of objects. Examples: `String`, `Arrays`, `Classes`, `Interfaces`.

Q9 What is the double colon (`::`) operator in Java?

RECOMMENDED RESPONSE:

"Introduced in Java 8, the `::` operator is known as the **Method Reference operator**. It is a compact, readable syntax to refer to a method without executing it, acting as a shorthand for lambda expressions. Example: `System.out::println` instead of `(x) -> System.out.println(x)`."

Q10 What happens if you call `main()` function inside `main()` function in Java/C?

RECOMMENDED RESPONSE:

"Calling `main()` inside `main()` creates a direct recursive call. Without a base termination condition, it will continuously push stack frames onto the call stack until memory is exhausted, throwing a `java.lang.StackOverflowError` in Java or causing a Segmentation Fault / Stack Overflow in C."

Q11 What is the difference between Java and JavaScript?

RECOMMENDED COMPARISON:

- **Java:** Strongly typed, compiled OOP language that runs on the Java Virtual Machine (JVM). Used for enterprise backends, Android, and desktop apps.
- **JavaScript:** Dynamically typed, interpreted/JIT scripting language that executes in browser engines (or Node.js runtime). Primarily used for interactive web applications.

Q12 What is the difference between List and Tuple in Python?

RECOMMENDED COMPARISON:

- **List:** Mutable (elements can be modified/added/removed after creation). Enclosed in square brackets `[1, 2, 3]`. Slower execution and consumes slightly more memory.
- **Tuple:** Immutable (read-only once defined). Enclosed in parentheses `(1, 2, 3)`. Faster execution, memory efficient, and usable as dictionary keys.

Q13 Why is the `auto` keyword used in C?

RECOMMENDED RESPONSE:

"In C language, `auto` stands for Automatic Storage Class. It is the default storage class for all local variables declared inside a function block. It specifies that memory is allocated automatically upon entry to the function and deallocated upon exit. (In modern C++, `auto` is used for automatic type inference)."

CATEGORY 4: PRACTICAL CODING & ALGORITHMIC SOLUTIONS

Q14 **Write code to check if a String/Sentence is a Palindrome (handling spaces, special characters, and case sensitivity).**

EXECUTABLE JAVA SOLUTION:

```
public class PalindromeCheck {
    public static boolean isPalindrome(String s) {
        // Remove non-alphanumeric characters and convert to lowercase
        String cleanStr = s.replaceAll("[^a-zA-Z0-9]", "").toLowerCase();
        int left = 0, right = cleanStr.length() - 1;

        while (left < right) {
            if (cleanStr.charAt(left) != cleanStr.charAt(right)) {
                return false;
            }
            left++;
            right--;
        }
        return true;
    }

    public static void main(String[] args) {
        String test = "A man, a plan, a canal: Panama";
        System.out.println("Is Palindrome: " + isPalindrome(test)); // Returns true
    }
}
```

Q15 **Write a program to find the largest/longest word in a given sentence and reverse it.**

EXECUTABLE JAVA SOLUTION:

```
public class LongestWordReverse {
    public static String findAndReverseLongest(String sentence) {
        String[] words = sentence.split("\s+");
        String longest = "";

        for (String word : words) {
            // Clean punctuation if needed
            String cleanWord = word.replaceAll("[^a-zA-Z]", "");
            if (cleanWord.length() > longest.length()) {
                longest = cleanWord;
            }
        }

        // Reverse the longest word
        return new StringBuilder(longest).reverse().toString();
    }

    public static void main(String[] args) {
        String sentence = "Mphasis campus placement interview preparation";
        System.out.println("Reversed Longest: " + findAndReverseLongest(sentence));
    }
}
```

Q16 Write a Java program to find the second largest element in an array.

EXECUTABLE JAVA SOLUTION:

```java
public class SecondLargest {
    public static int getSecondLargest(int[] arr) {
        if (arr == null || arr.length < 2) return -1;

        int largest = Integer.MIN_VALUE;
        int secondLargest = Integer.MIN_VALUE;

        for (int num : arr) {
            if (num > largest) {
                secondLargest = largest;
                largest = num;
            } else if (num > secondLargest && num != largest) {
                secondLargest = num;
            }
        }
        return secondLargest;
    }

    public static void main(String[] args) {
        int[] numbers = {12, 35, 1, 10, 34, 1};
        System.out.println("Second Largest: " + getSecondLargest(numbers)); // Output: 34
    }
}
```

Q17 Write pseudo-code for Recursion to calculate factorial and find string length.

PSEUDO-CODE:

```
// 1. Recursive Factorial
FUNCTION factorial(n):
    IF n <= 1 THEN
        RETURN 1
    ELSE
        RETURN n * factorial(n - 1)
END FUNCTION

// 2. Recursive String Length
FUNCTION getLength(str):
    IF str EQUALS "" THEN
        RETURN 0
    ELSE
        RETURN 1 + getLength(substring of str from index 1 to end)
END FUNCTION
```

CATEGORY 5: DATA STRUCTURES & ALGORITHMS (DSA)

Q18 What is a Data Structure? Explain Linear vs Non-Linear.

RECOMMENDED RESPONSE:

"A Data Structure is a specialized format for organizing, processing, retrieving, and storing data efficiently in computer memory.

- **Linear Data Structures:** Elements are arranged sequentially where each element is connected to its previous and next elements. Examples: Arrays, Linked Lists, Stacks, Queues.
- **Non-Linear Data Structures:** Elements are arranged hierarchically or interconnected across multiple levels. Examples: Trees, Binary Search Trees, Graphs, Hash Tables.

"

Q19 Explain Merge Sort vs Bubble Sort (Algorithm logic and Complexity).

RECOMMENDED COMPARISON:

- **Merge Sort:** Divide-and-Conquer algorithm. Recursively divides the array into halves, sorts sub-arrays, and merges them back. Time Complexity: `O(N log N)` across best, average, and worst cases. Space Complexity: `O(N)`.
- **Bubble Sort:** Comparison-based algorithm. Repeatedly steps through the list, compares adjacent elements, and swaps them if they are in wrong order. Time Complexity: `O(N^2)` worst/average, `O(N)` best case. Space Complexity: `O(1)`.

Q20 How do you perform Merge Sort using a Linked List?

RECOMMENDED EXPLANATION:

"Merge sort is the preferred algorithm for sorting linked lists because linked lists allow `O(1)` pointer rearrangements during the merge step without requiring extra array allocation space:

1. **Find Middle:** Use Fast and Slow pointer approach (Floyd's cycle concept) to find the midpoint of the linked list.
2. **Divide:** Split the linked list into two independent sub-lists at the middle pointer.
3. **Conquer:** Recursively call Merge Sort on both sub-lists.
4. **Merge:** Combine the two sorted linked lists using a standard two-pointer merging technique."

CATEGORY 6: DATABASE MANAGEMENT SYSTEMS (DBMS) & SQL

Q21 What is the difference between TRUNCATE, DELETE, and DROP commands in SQL?

RECOMMENDED COMPARISON TABLE/SUMMARY:

- **DELETE (DML):** Removes specific or all rows based on a `WHERE` clause. Operations are logged row-by-row and can be rolled back inside transactions. Triggers fire.
- **TRUNCATE (DDL):** Removes all rows from a table by deallocating data pages. Faster than DELETE, cannot use `WHERE` clause, resets identity seeds, fires no delete triggers.
- **DROP (DDL):** Completely removes the table structure, data, indexes, and permissions permanently from the database catalog.

Q22 What are DDL, DML, DCL, and TCL commands? Give examples.

RECOMMENDED RESPONSE:

- **DDL (Data Definition Language):** Defines database schema/structure. Commands: `CREATE`, `ALTER`, `DROP`, `TRUNCATE`.
- **DML (Data Manipulation Language):** Manages data records. Commands: `SELECT`, `INSERT`, `UPDATE`, `DELETE`.
- **DCL (Data Control Language):** Manages user permissions/security. Commands: `GRANT`, `REVOKE`.
- **TCL (Transaction Control Language):** Manages transactional state. Commands: `COMMIT`, `ROLLBACK`, `SAVEPOINT`.

Q23 What are Joins in SQL? Explain Inner Join, Left Join, Right Join, and Full Join with Queries.

RECOMMENDED SQL QUERIES & EXPLANATIONS:

"SQL Joins combine columns from one or more tables based on a related common column:

- **INNER JOIN:** Returns records that have matching values in both tables.
- **LEFT JOIN:** Returns all records from left table, and matched records from right table.

```
-- Count employees working in each department using INNER JOIN & GROUP BY
SELECT d.department_name, COUNT(e.emp_id) AS total_employees
FROM Department d
INNER JOIN Employee e ON d.dept_id = e.dept_id
GROUP BY d.department_name;
```

"

Q24 Write an SQL query to retrieve the current date.

RECOMMENDED QUERIES ACROSS SQL DIALECTS:

```
-- MySQL / PostgreSQL
SELECT CURRENT_DATE;

-- SQL Server (T-SQL)
SELECT GETDATE();

-- Oracle
SELECT SYSDATE FROM DUAL;
```

CATEGORY 7: SOFTWARE ENGINEERING, SDLC, GIT & WEB TECH

Q25 Explain SDLC phases and compare Waterfall vs Agile models.

RECOMMENDED RESPONSE:

"SDLC phases include Requirement Analysis, System Design, Implementation (Coding), Testing, Deployment, and Maintenance.

- **Waterfall Model:** Linear, sequential approach. Requirements must be fixed upfront. Phase transitions occur only after previous phase completion. High risk for changing needs.
- **Agile Model:** Iterative and incremental approach. Software delivered in short 2-4 week sprints. Embraces requirement changes, continuous feedback, and rapid releases."

Q26 What is Git Rebase, and how does it differ from Git Merge? Name key Git commands.

RECOMMENDED RESPONSE:

- **Git Merge:** Combines changes from target branch into current branch creating a new 'merge commit'. Preserves complete historic commit topology.
- **Git Rebase:** Re-applies commits from feature branch on top of tip of target branch, creating a clean, linear commit history.
- **Key Git Commands:** `git init`, `git clone`, `git status`, `git add .`, `git commit -m ""`, `git push`, `git pull`, `git branch`.

Q27 What is HTTP vs HTTPS? What are Hooks in React?

RECOMMENDED RESPONSE:

- **HTTP vs HTTPS:** HTTP (HyperText Transfer Protocol) transmits plaintext over port 80. HTTPS encrypts communication using TLS/SSL over port 443, securing data against eavesdropping.
- **React Hooks:** Functions introduced in React 16.8 that allow functional components to manage state and lifecycle features without writing ES6 classes (e.g., `useState`, `useEffect`).

CATEGORY 8: SCENARIO-BASED, LOGIC & STRESS INTERVIEW SCENARIOS

Q28 Probability Scenario: If I place 100 colored balls in front of you and ask you to pick 1 red ball, what is the probability of getting red?

RECOMMENDED RESPONSE:

"To calculate the exact probability, I need to know the total number of red balls (R) in the set of 100. The probability formula is `P(Red) = Number of Red Balls (R) / Total Balls (100)`. If the distribution is uniform across say 5 colors (20 of each), P(Red) = 20/100 = 0.2 (20%). If there are zero red balls, probability is 0."

Q29 Scenario: Your colleague delivers a required dataset on Thursday, but your project deadline is Saturday (2-week task compressed). How do you handle it?

RECOMMENDED STRATEGY:

"First, I would not panic. I would immediately run automated data validation scripts on the dataset received to check for quality/missing values. Second, I would prioritize building the core Minimum Viable Product (MVP) pipeline first rather than optional features. Third, I would communicate transparently with my team lead on Thursday evening, detailing the critical path and working dedicated hours to ensure the core deliverable is ready by Saturday."

Q30 Stress Test Scenario: The interviewer asks you to stand up, move back, and deliver an unscripted welcome speech to the room.

RECOMMENDED STRATEGY:

"This tests presence of mind, confidence under pressure, and public speaking willingness. Stand up immediately without hesitation, maintain good posture, project your voice clearly, and deliver a warm 45-second greeting welcoming everyone, expressing enthusiasm for the platform, and thanking the organizers."

> *Candidate Insight: As recorded in review data, candidates who stepped up without hesitation passed this stress evaluation immediately!*

Q31 How do you handle technical rejection or software component failure?

RECOMMENDED STRATEGY:

"I view rejection or software failure not as a personal setback, but as valuable diagnostic feedback. When software fails, I systematically check application logs, trace stack traces, isolate root causes, and write regression tests. When facing personal rejection, I ask for feedback, identify skill gaps, and build a structured learning plan to improve."